> ¿Cómo conseguir que una señal analógica sea muestreada a intervalos de tiempo lo más uniformes posible para que luego pueda ser procesada y reconstruida correctamente?

> 🖉 Nota
>
> La Raspberry Pi Pico es especialmente adecuada porque permite mostrar experimentalmente que "leer un ADC cada cierto tiempo" no necesariamente significa "muestrear uniformemente".

Adquisición de datos en tiempo real con Raspberry Pi Pico

Tema

Polling, interrupciones y multitarea en MicroPython aplicados a la adquisición de señales analógicas

Plataforma

- Raspberry Pi Pico / Pico W
- MicroPython
- ADC del RP2040
- PC para visualización y procesamiento de datos

Duración sugerida

3 a 4 horas, divididas en:

- 45–60 min: fundamentos teóricos
- 30 min: demostración conceptual
- 90–120 min: trabajos prácticos
- 30 min: análisis de resultados y conclusiones

---

1. Objetivos de la clase

Al finalizar la actividad, el estudiante debería poder:

1. Comprender las diferencias entre:
   - polling,

    - interrupciones,
    - multitarea/hilos.
2. Implementar las tres estrategias en MicroPython.
3. Utilizar el ADC del RP2040 para adquirir una señal analógica.
4. Comprender la diferencia entre:

   frecuencia de adquisición programada

   y

   frecuencia de muestreo efectiva.
5. Medir experimentalmente la separación temporal entre muestras.
6. Identificar el **jitter temporal** producido por el software.
7. Comprender por qué un muestreo no uniforme puede afectar el procesamiento digital posterior.
8. Relacionar el muestreo temporal con:
    - frecuencia de Nyquist,
    - aliasing,
    - reconstrucción,
    - procesamiento digital de señales.
9. Comparar experimentalmente polling, interrupciones y multitarea.

---

2. Concepto central de la clase

Conviene que toda la clase gire alrededor de esta pregunta:

> **¿Qué tan homogéneo es realmente nuestro muestreo cuando ejecutamos un programa que dice `sleep_ms(10)`?**

Por ejemplo, si pretendemos adquirir:

$$f_s = 100\ Hz$$

entonces:

$$TT_s = \frac{1}{f_s} = 10\ ms$$

Idealmente deberíamos obtener:

$$t_0,\quad t_0 + 10ms,\quad t_0 + 20ms,\quad t_0 + 30ms, \ldots$$

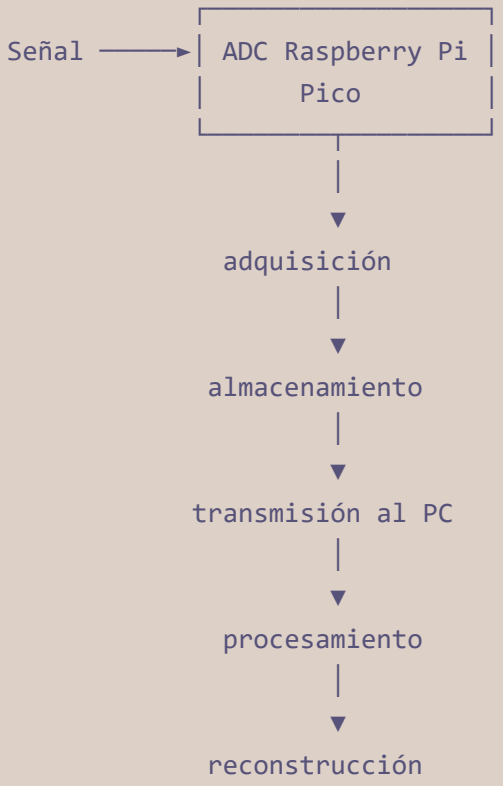
Pero en un sistema real podemos obtener:

$$t_0,\quad t_0 + 10.3ms,\quad t_0 + 19.7ms,\quad t_0 + 30.8ms, \ldots$$

La diferencia entre el instante real y el instante ideal constituye una forma de **error temporal o jitter**.

3. Introducción al sistema de adquisición

Podemos plantear inicialmente el sistema:



El ADC convierte:

$$x(t) \to x[n]$$

donde:

$$x[n] = x(t_n)$$

Para un muestreo uniforme:

$$t_n = nT_s$$

y por lo tanto:

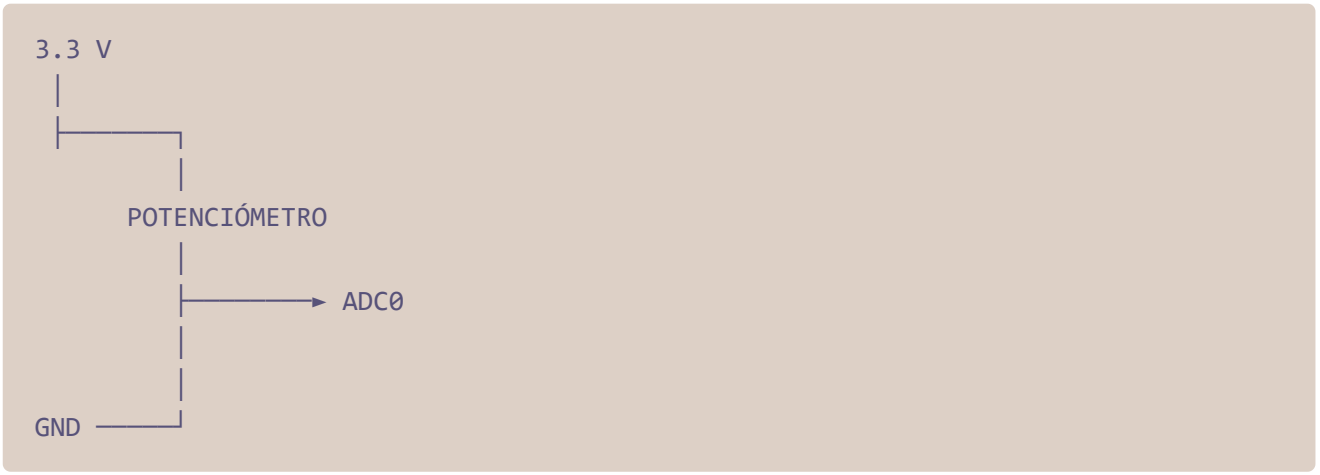
$$x[n] = x(nT_s)$$

Este detalle será fundamental durante toda la clase.

4. Primera parte práctica: ADC del Pico

Comenzaría con un ejemplo extremadamente sencillo.

Por ejemplo, conectar un potenciómetro:



En MicroPython:

```
from machine import ADC
import time

adc = ADC(26)

while True:
    valor = adc.read_u16()
    print(valor)
    time.sleep_ms(100)
```

El objetivo aquí no es todavía hacer una adquisición correcta, sino introducir el problema.

Preguntar a los estudiantes:

> ¿Estamos tomando una muestra exactamente cada 100 ms?

La respuesta esperada es:

No necesariamente.

---

5. Medición del tiempo real de muestreo

Este sería uno de los experimentos más importantes.

Podemos medir el instante de cada adquisición:

```
from machine import ADC
import time

adc = ADC(26)

while True:
    t = time.ticks_us()
```

```
    x = adc.read_u16()

    print(t, x)

    time.sleep_ms(100)
```

Luego en el PC se pueden calcular:

$$\Delta t_n = t_n - t_{n-1}$$

y observar:

```
Muestra       Δt [ms]

1             100.2
2              99.8
3             100.4
4              99.7
5             100.1
6             101.0
...
```

La discusión pasa entonces de:

> "El programa muestrea a 10 Hz"

a:

> "El programa intenta muestrear a 10 Hz, pero ¿cuál es la distribución real de los intervalos de muestreo?"

---

6. Polling

Concepto

El polling consiste en que el procesador pregunta periódicamente si debe realizar una determinada tarea.

Por ejemplo:

```
while True:

    if tiempo_de_muestrear:
        adquirir()

    if hay_datos_para_transmitir:
```

```
        transmitir()

    if boton_presionado:
        procesar_boton()
```

El procesador está continuamente ejecutando el programa.

Ventaja

Es:

- simple,
- fácil de comprender,
- sencillo de depurar.

Desventaja

El tiempo de respuesta depende de cuánto tarda el programa en recorrer el ciclo.

---

7. TP 1 — Adquisición mediante polling

La primera implementación puede ser:

```
from machine import ADC
import time

adc = ADC(26)

Ts_us = 100_000

t_proxima = time.ticks_us()

while True:

    t_proxima = time.ticks_add(t_proxima, Ts_us)

    x = adc.read_u16()
    t = time.ticks_us()

    print(t, x)

    while time.ticks_diff(t_proxima, time.ticks_us()) > 0:
        pass
```

Esto permite introducir una cuestión muy importante:

¿Dónde está el tiempo?

No solamente en:

```
time.sleep_ms()
```

sino en:

- lectura ADC,
- cálculos,
- `print()`,
- comunicación serie,
- procesamiento,
- interrupciones,
- sistema operativo interno de MicroPython.

---

8. Un experimento muy interesante

Modificar el programa:

```
x = adc.read_u16()

print(t, x)
```

y posteriormente:

```
x = adc.read_u16()

# no imprimir
```

Comparar los intervalos:

$$\Delta t_n$$

Esto permite demostrar experimentalmente que la comunicación serie puede introducir una perturbación significativa en el proceso de adquisición.

Esta idea es fundamental en sistemas de adquisición:

> No conviene mezclar directamente adquisición crítica en tiempo con tareas cuyo tiempo de ejecución es variable.

---

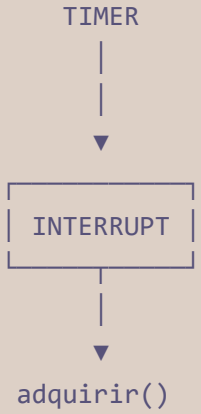
9. Interrupciones

Después de estudiar polling, introducir:

> ¿Podemos hacer que el procesador no tenga que preguntar continuamente si llegó el momento de adquirir?

Aquí aparecen las interrupciones.

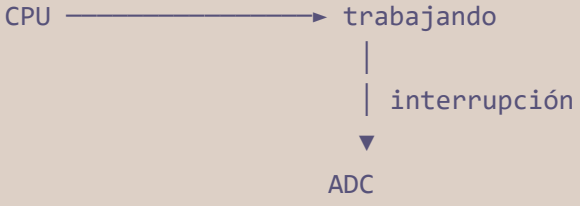
Conceptualmente:



En lugar de:

```
CPU ──► ¿ya?
CPU ──► ¿ya?
CPU ──► ¿ya?
CPU ──► ¿ya?
CPU ──► ¿ya?
```

tenemos:



---

10. Concepto importante: ISR

En MicroPython una rutina de interrupción debe ser:

- corta,
- rápida,
- determinista en lo posible.

Por ejemplo, conceptualmente:

```
def timer_callback(timer):
    global flag
    flag = True
```

Y el programa principal:

```
while True:

    if flag:
        flag = False

        x = adc.read_u16()
```

Esto introduce una arquitectura muy interesante:

```
                TIMER
                  |
                  ▼
            interrupción
                  |
                  ▼
                FLAG
                  |
                  ▼
          programa principal
                  |
                  ▼
                 ADC
```

La interrupción no necesariamente debe hacer todo el trabajo.

Puede simplemente generar un evento.

---

11. TP 2 — Muestreo sincronizado mediante Timer

Utilizar:

```
from machine import ADC, Timer
import time

adc = ADC(26)

flag = False

def timer_callback(timer):
    global flag
```

```
    flag = True

timer = Timer()
timer.init(
    period=100,
    mode=Timer.PERIODIC,
    callback=timer_callback
)

while True:

    if flag:
        flag = False

        t = time.ticks_us()
        x = adc.read_u16()

        print(t, x)
```

Aquí hay una discusión muy interesante:

¿Es ahora perfecto el tiempo de muestreo?

No.

Pero hemos separado:

generación del evento temporal

de

procesamiento de la muestra.

Esto permite discutir las limitaciones de MicroPython y de un sistema no determinista.

---

12. Una cuestión fundamental: ISR ≠ sistema de tiempo real duro

Es importante evitar que el estudiante concluya:

> "Con una interrupción tenemos tiempo perfecto."

No necesariamente.

Hay:

- latencia,
- jitter,

- interrupciones internas,
- garbage collector,
- ejecución del intérprete,
- operaciones que no son deterministas.

Por eso conviene introducir:

$$t_n = nT_s + \epsilon_n$$

donde:

$$\epsilon_n$$

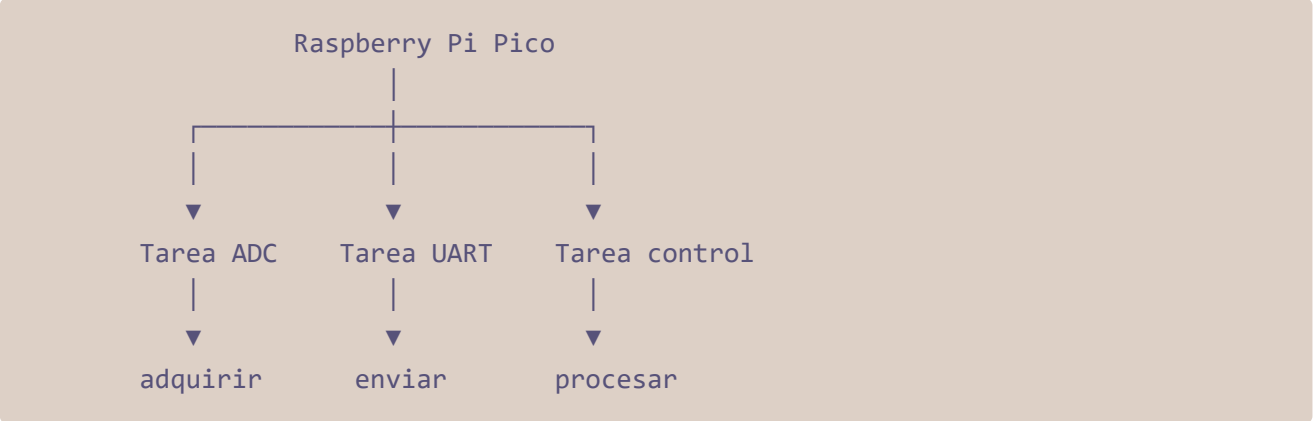
representa el error temporal de cada muestra.

---

13. Multitarea / multihilo

Finalmente introducimos el concepto de tareas concurrentes.

Podemos imaginar:



En MicroPython se puede introducir `_thread`.

Por ejemplo:

```
import _thread
import time

def tarea_1():
    while True:
        print("Adquisición")
        time.sleep_ms(100)

_thread.start_new_thread(tarea_1, ())

while True:
    print("Proceso principal")
    time.sleep_ms(250)
```

Esto permite mostrar que las tareas pueden ejecutarse concurrentemente.

---

14. Pero aparece un nuevo problema

Supongamos:

```
Hilo 1 → adquisición
Hilo 2 → procesamiento
```

Ambos necesitan acceder a:

```
buffer[]
```

Entonces tenemos una condición de carrera.

Por ejemplo:



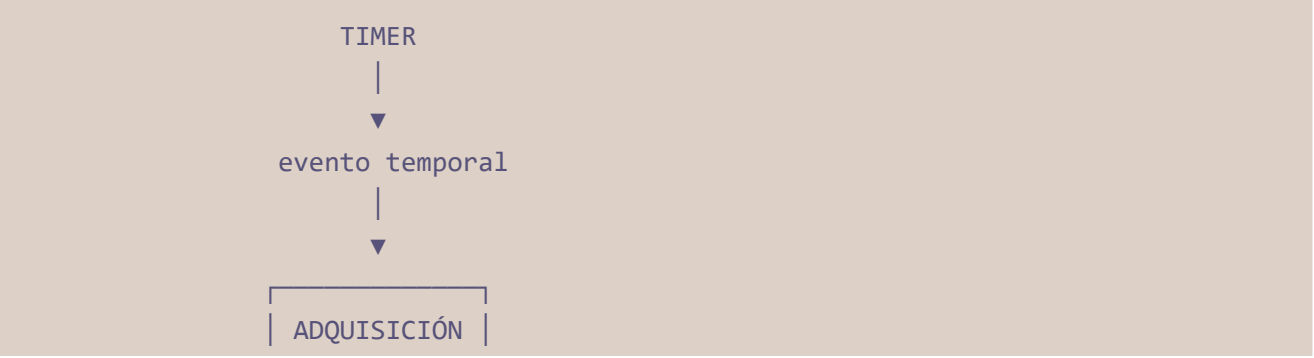
Aquí aparecen conceptos como:

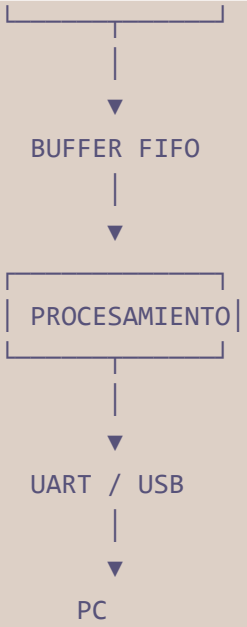
- recursos compartidos,
- exclusión mutua,
- locks,
- sincronización,
- productor/consumidor.

---

15. Arquitectura recomendada para la práctica

Una arquitectura muy didáctica sería:





Esto permite separar claramente:

Tiempo crítico

```
adquisición
```

de:

Tiempo no crítico

```
procesamiento
transmisión
visualización
```

---

16. El concepto de buffer

Aquí introduciría uno de los conceptos más importantes de la clase.

En lugar de:

```
ADC → print()
ADC → print()
ADC → print()
```

usar:

```
ADC → BUFFER
ADC → BUFFER
ADC → BUFFER
```

```
ADC → BUFFER
            ↓
         enviar
            ↓
            PC
```

Por ejemplo:

```
buffer = []

buffer.append(x)
```

Aunque para una implementación más seria conviene trabajar con un buffer circular o una estructura FIFO.

---

17. TP 3 — Adquisición y procesamiento concurrentes

El objetivo sería crear:

Tarea 1

Adquisición:

```
100 Hz
```

Tarea 2

Procesamiento:

```
promedio
máximo
mínimo
RMS
```

Tarea 3

Comunicación:

```
enviar datos al PC
```

De esta manera los estudiantes pueden comprobar que:

> La adquisición no debería depender directamente de la velocidad de procesamiento o transmisión.

---

18. Experimento fundamental: señal sinusoidal

Una vez implementados los tres métodos, utilizar una señal conocida:

$$x(t) = A\sin(2\pi f_0 t)$$

Por ejemplo:

$$f_0 = 5\,Hz$$

y:

$$f_s = 100\,Hz$$

Entonces:

$$\frac{f_s}{f_0} = 20$$

Tenemos aproximadamente 20 muestras por período.

Visualmente:

```
Señal analógica

      / \       / \
     /   \     /   \
-----/-----\---/-----\----
    /       \ /       \
```

y las muestras:

```
   •   •   •   •
  •             •
 •               •
```

---

19. Reconstrucción

Luego se puede enviar al PC:

```
t,x
0.000,2048
0.010,2350
```

```
0.020,2650
...
```

y reconstruir mediante Python:

```
plt.plot(t, x, '.-')
```

El objetivo no es solamente "hacer un gráfico bonito".

La pregunta debe ser:

> ¿Qué ocurre si los tiempos de las muestras no son uniformes pero suponemos que sí lo son?

---

20. Comparación entre muestreo uniforme y no uniforme

Podemos plantear:

Caso ideal

$$t_n = nT_s$$

Caso real

$$t_n = nT_s + \epsilon_n$$

Si luego procesamos suponiendo:

$$t_n = nT_s$$

estamos ignorando:

$$\epsilon_n$$

Esto puede afectar:

- estimación de frecuencia,
- FFT,
- fase,
- filtrado,
- integración,
- derivación,
- reconstrucción.

---

21. Relación con Nyquist

Una vez comprendido el problema temporal, introducir:

$$f_s > 2f_{max}$$

como condición básica del teorema de muestreo.

Pero inmediatamente agregar:

> **Cumplir Nyquist no garantiza por sí solo una adquisición de buena calidad.**

También debemos considerar:

- jitter,
- resolución ADC,
- ruido,
- ancho de banda,
- estabilidad del reloj,
- cuantización,
- anti-aliasing.

Esto da pie a una discusión mucho más realista de un sistema de adquisición.

---

22. Comparación final

Una tabla como esta puede cerrar la clase:

Característica	Polling	Interrupciones	Multitarea
Complejidad	Baja	Media	Alta
Facilidad de programación	Alta	Media	Baja
Respuesta a eventos	Media	Alta	Alta
Control temporal	Limitado	Bueno	Depende de implementación
Recursos compartidos	Simple	Moderado	Complejo
Jitter	Puede ser elevado	Menor	Variable
Depuración	Fácil	Media	Difícil
Adecuado para adquisición	Sí, simple	**Muy bueno**	Sí, con cuidado
Procesamiento paralelo	No	No necesariamente	Sí
Riesgo de condiciones de	Bajo	Bajo/medio	**Alto**

Característica	Polling	Interrupciones	Multitarea
carrera			

---

23. Secuencia pedagógica que recomiendo

Yo organizaría la clase en cuatro experiencias consecutivas:

Experiencia 1 — Polling

```
ADC
 ↓
while
 ↓
sleep
 ↓
ADC
```

Medir:

$$\Delta t$$

Experiencia 2 — Polling + procesamiento

Agregar deliberadamente una carga:

```
procesar()
```

y observar qué ocurre con:

$$\Delta t$$

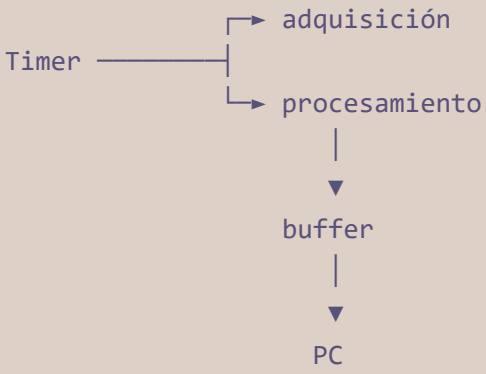
Esto demuestra que el tiempo de ejecución del programa afecta la adquisición.

Experiencia 3 — Timer + interrupción

```
Timer
 ↓
ISR
 ↓
flag
 ↓
ADC
 ↓
buffer
```

Comparar el jitter con el caso anterior.

Experiencia 4 — Adquisición + procesamiento concurrente



Analizar:

- estabilidad temporal,
- utilización de CPU,
- pérdida de muestras,
- comportamiento del buffer.

---

24. Una actividad particularmente interesante

Para que la diferencia entre los métodos sea experimentalmente evidente, introduciría una tarea que consuma tiempo.

Por ejemplo:

```
def procesamiento_pesado():
    suma = 0

    for i in range(10000):
        suma += i*i

    return suma
```

Luego hacer:

```
Adquisición
     +
procesamiento pesado
```

y observar:

$$\Delta t_n$$

Después separar las tareas.

El estudiante debería descubrir experimentalmente:

> Una tarea lenta o de duración variable puede perturbar el muestreo cuando adquisición y procesamiento están acoplados.

Esta conclusión es mucho más valiosa que simplemente explicar teóricamente qué es un hilo.

---

25. Proyecto integrador

Como cierre, propondría un pequeño sistema de adquisición:

Entrada

ADC0:

```
señal analógica
```

Frecuencia

$$f_s = 100\,Hz$$

Adquisición

```
Timer → evento → ADC
```

Buffer

Por ejemplo:

```
500 muestras
```

Procesamiento

Calcular:

$$RMS = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x_n^2}$$

Transmisión

Enviar al PC:

```
n,t,x
```

En PC

Graficar:

1. $x(t)$
2. Δt_n
3. histograma de Δt_n
4. error temporal:

$$e_n = \Delta t_n - T_s$$

---

26. Preguntas para el informe

Podría finalizar con preguntas como:

1. ¿Cuál fue el período de muestreo solicitado?
2. ¿Cuál fue el período promedio obtenido?
3. ¿Cuál fue el máximo y mínimo Δt\Delta t?
4. ¿Cuál fue el jitter?
5. ¿Qué estrategia presentó mayor dispersión temporal?
6. ¿Cómo afecta `print()` al proceso de adquisición?
7. ¿Por qué una ISR debería ser corta?
8. ¿Qué problemas aparecen cuando dos hilos utilizan el mismo buffer?
9. ¿Por qué conviene desacoplar adquisición y procesamiento?
10. ¿Qué sucede si fs<2fmaxf$s < 2f${max}?
11. ¿Qué sucede si fsf_s cumple Nyquist pero existe un jitter importante?
12. ¿Es MicroPython adecuado para cualquier sistema de adquisición?

Concepto final que debería quedar en los estudiantes

Me parece especialmente importante que la clase no termine simplemente con:

> **Polling vs. interrupciones vs. threads.**

Sino con una conclusión de ingeniería:

> **En un sistema de adquisición, el dato no está definido solamente por su valor x[n]x[n], sino también por el instante tnt_n en que fue adquirido.**

Por eso:

$$\boxed{x[n] = x(t_n)}$$

y, para un muestreo uniforme:

$$\boxed{t_n = t_0 + nT_s}$$

Mientras que en un sistema real:

$$\boxed{t_n = t_0 + nT_s + \epsilon_n}$$

El objetivo de la arquitectura de software es, entre otras cosas, **hacer que ϵ_n sea lo suficientemente pequeño y controlado para que no comprometa el procesamiento posterior**.

Si querés convertir esto en una **clase lista para dictar**, el siguiente paso que recomiendo es armarla como un **apunte completo + presentación conceptual + 3/4 TP progresivos**, incluyendo los **programas MicroPython completos**, el circuito del potenciómetro/señal analógica, generación de una señal de prueba, captura de datos por USB y un **script Python para la PC que calcule automáticamente T_s, jitter, histograma y FFT**.